From Image Classification to Threat Reasoning:

Building an End-to-End Aircraft Identification and Decision Support Pipeline

Overview

This project started with a simple question:

How can we improve aircraft identification: making it faster, more accurate, and enabling a more objective threat assessment—under operational time pressure??

The Operational Setting: Naval officers often have only seconds to respond following a radar return or visual confirmation, during which an aircraft must be identified and its threat potential assessed. In such environments, ground truth is rarely available in real time, and labels must be curated from incomplete, noisy, and evolving information sources. Misclassification carries asymmetric risks: failing to identify a genuine threat (*false negative*) may lead to dangerous complacency, while incorrectly classifying a friendly or neutral aircraft as hostile (*false positive*) can result in catastrophic escalation or fratricide. As a result, the system prioritizes timely, explainable, and uncertainty-aware outputs over absolute classification certainty.

Inspired by how naval and air combat information officers operate in real-world settings, the goal evolved into building a system that does more than just classify an image. Instead, it aims to:

- **Identify an aircraft from an image:** The type of aircraft.
- **Quantify confidence in that identification:** How sure are we that it is this aircraft.
- **Retrieve structured technical specifications:** What are the combat capabilities of this aircraft.
- **Reason about threat level under uncertainty:** Given the combat capabilities, how much of a threat does it pose to ourself.
- **Recommend sensible next actions:** What's the best course of action given all the information we have(includes **HUMAN** in the loop here)

This document walks through how that system was built end-to-end, including what went wrong along the way.

High-Level Pipeline

At a high level, the pipeline looks like this:

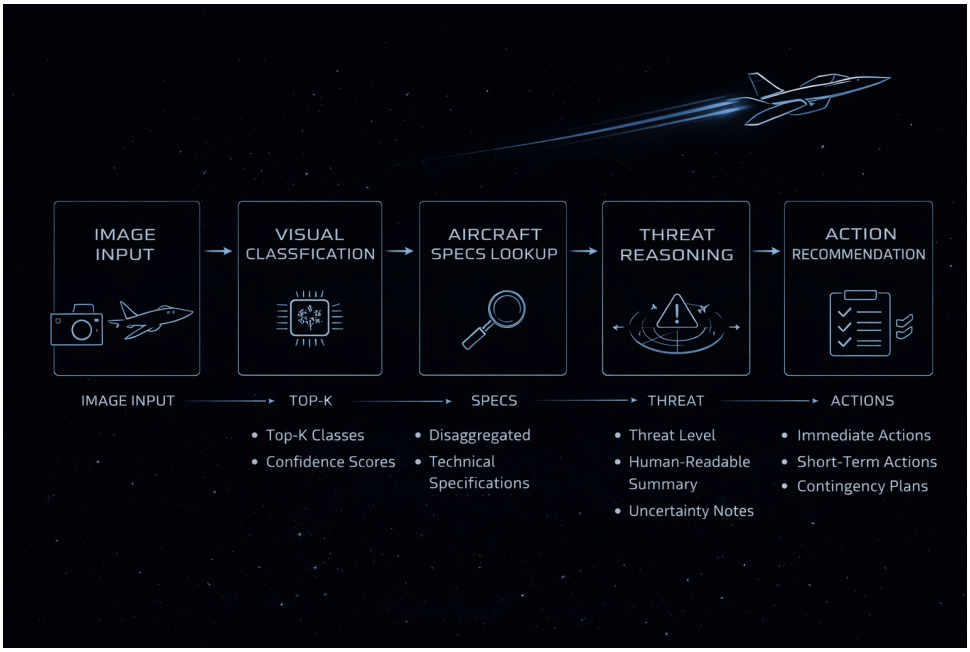

Figure 1: Operational Pipeline

Dataset and Early Challenges

Probably spent the most time over here (If not looped back and redid the data collection due to issues we will discuss later on). Used selenium to automate scraping from wikipedia

headlessly because all the other options for looking up required api keys and unneccessary overcomplications and paywalls.

We collect the names of aircrafts and append it to a csv file and use the csv file then to collect the corresponding images after that to create our own dataset (*In my head it was perfect, however given that I've enumerated the list of Challenges tells you something doesn't it?*).

Challenge 1: Initial Wikipedia Scraping

Approach: The initial data collection performed surface-level scraping of Wikipedia, extracting primary links and saving them to a CSV. This method was designed to avoid deep link navigation and an excessive number of dataset classes.

Drawback: This resulted in only high-level category links (e.g., "aircraft from the 1970s," "aircraft from the 1980s") without differentiating between list pages and individual aircraft pages.

Solution: Integrated Ollama into the pipeline to classify whether a given Wikipedia page represented a list of aircraft or a specific aircraft model.

Outcome: Gathered approximately 1700 aircraft classes. However, the dataset included significant noise from unrelated pages such as aircraft accidents and explosions.

Challenge 2: Revised Wikipedia Scraping for Dataset Cleaning

Approach: Refined the Ollama query to explicitly filter out noise entries like accidents, explosions, and incomplete aircraft pages.

Drawback: The more aggressive link crawling began including pages for **prototypes and experimental weapons**, which were not desired for the target operational dataset.

Solution: Enhanced the Ollama filter to also exclude prototypes and experimental aircraft.

Outcome: Reduced the dataset to roughly 1300 aircraft classes, though it still contained **non-operational aircraft**.

Challenge 3: Image Collection

Approach: Used a naive method to gather images from common sources like Wikimedia and Google search results.

Drawback: The collected images were often irrelevant (non-aircraft), with a high prevalence of **accident and explosion scenes**.

Solution: Implemented a soft filtering step using HTML scraping and the *alt* text from page sources to better identify relevant images.

Outcome: Successfully collected images for approximately 800 aircraft classes.

Challenge 4: Insufficient Images per Class

Approach: Many aircraft classes, particularly from the early aviation era, had few or no available images due to limited documentation.

Drawback: This scarcity drastically reduced the usable number of classes for model training.

Solution: Applied a minimum image threshold and removed all aircraft classes below it, as preliminary models trained on sparse data performed poorly.

Outcome: Final dataset contained approximately **35 viable aircraft classes**.

Challenge 5: Final Data Quality and Structural Integrity

Approach: A final audit of the dataset revealed that one class (PN-1) contained significant noise and irrelevant images.

Problem: Removing this class altered the label space, which caused a classifier head dimension mismatch in the trained model.

Solution: The PN-1 class was purged, necessitating a complete model retrain from scratch.

Outcome: This resulted in a final, clean dataset of 34 aircraft classes. The process underscored a critical lesson: any change to the label space has cascading effects and requires full retraining.

Once the dataset was fully vetted, it was then split 80:20 into train and val and metrics were measured for various models.

Model Training

Several architectures were evaluated, including metric-learning approaches and transformer-based models. Ultimately, a fine-tuned EfficientNet variant provided the best balance between performance, stability, and inference speed for this use case.

Key techniques used:

- **Class-balanced loss:** Adjusts the loss function to account for imbalanced class distributions, preventing the model from being biased toward the majority classes.
- **Label smoothing** – Replaces hard one-hot labels with smoothed targets (e.g., 0.9 for the true class, 0.1 for others) to reduce model overconfidence and improve generalization.
- **Freeze / unfreeze scheduling** – Initially freezes the backbone layers to stabilize early training, then progressively unfreezes them to allow fine-tuning of deeper features.
- **Explicit overfitting diagnostics (train–val gap tracking)** – Systematically monitors the performance gap between training and validation sets to detect overfitting early and adjust regularization or training duration.

Despite achieving high training accuracy early, validation accuracy plateaued around 40%. This gap was not ignored; it became a design constraint that influenced how much trust downstream reasoning should place on the classifier.

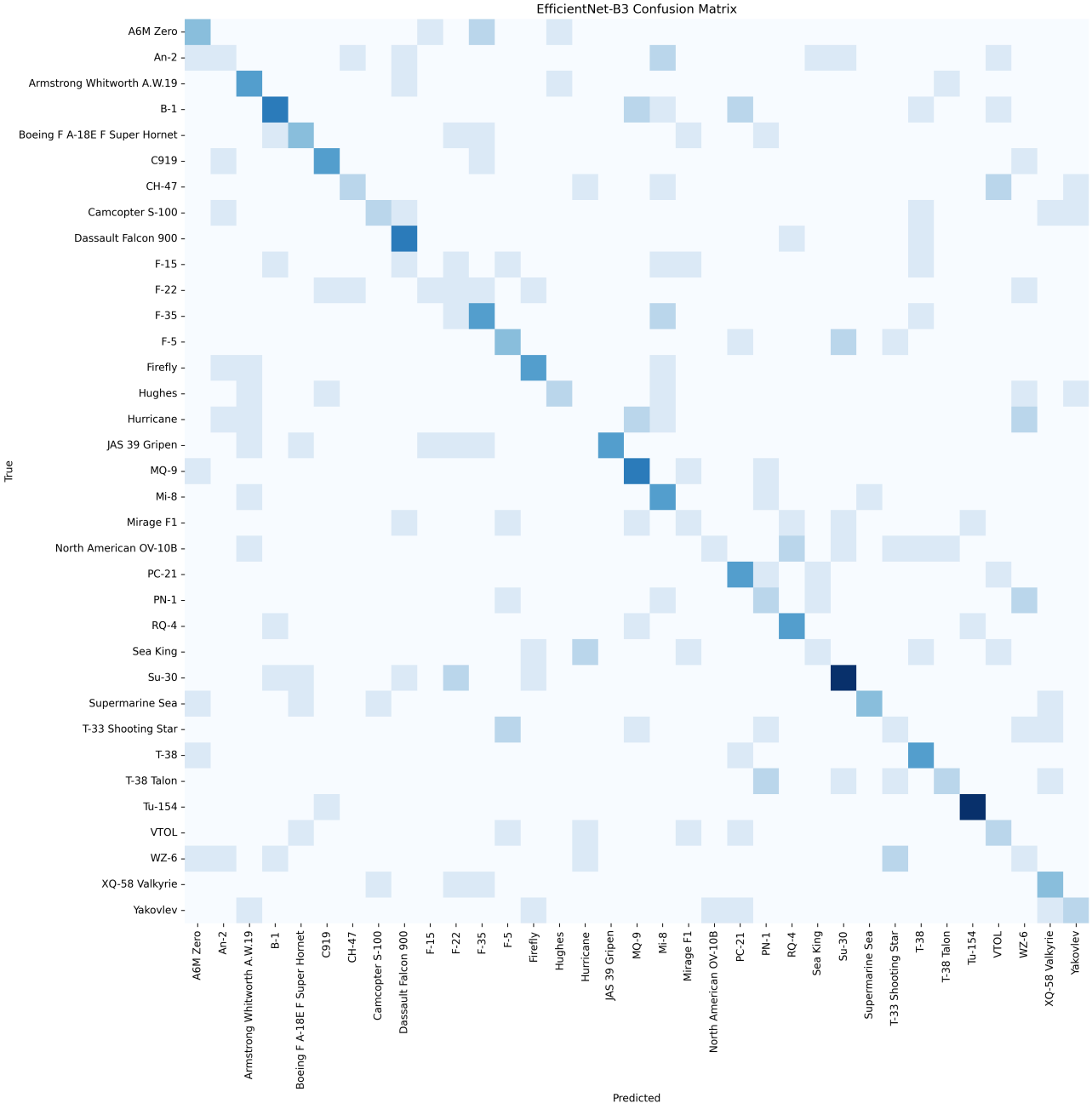

Figure 2: EfficientNet Confusion Matrix

Output

Figure 3: Output for F22 Raptor

In the evaluated example, an F-22 Raptor was classified as an F-15. While this constitutes a misclassification at the specific platform level, both aircraft belong to the same operational family of air-superiority fighters and share similar roles, performance envelopes, and threat characteristics. From an operational assessment perspective, this represents a partially correct identification rather than a complete failure. Overall classification accuracy peaked at approximately 41%. Despite extensive architectural experimentation and training refinements, further performance gains could not be achieved under the current constraints. This limitation is attributed primarily to the small size, class imbalance, and narrow visual diversity of the available training dataset, rather than deficiencies in model capacity or optimization strategy. Consequently, model performance appears to be data-limited rather than architecture-limited, indicating that meaningful improvements would require substantially broader and more representative training data rather than further model tuning. As such, the reported accuracy should be interpreted as a lower bound on achievable performance given the current dataset rather than an upper bound on the approach itself.

Top-K Classification Instead of Top-1

Rather than relying on a single predicted class, the inference pipeline uses a Top-K approach. This decision was deliberate:

- Real images are ambiguous
- Confidence matters more than correctness alone
- Downstream systems should degrade gracefully

Each Top-K output includes:

- Aircraft name
- Confidence score

Only structurally valid predictions are accepted, preventing silent failures from malformed outputs.

Aircraft Specification Database

To decouple visual perception from domain knowledge, aircraft specifications are extracted, normalized, and stored independently of the vision model.

For each aircraft class:

- A dedicated YAML specification file is generated and stored separately from the image dataset
- All quantitative attributes (e.g., speed, range, ceiling) are normalized into consistent numerical units
- Operational capabilities (e.g., beyond-visual-range combat, strike capability, electronic warfare) are represented as explicit boolean flags
- Missing or unavailable specifications are handled explicitly and preserved as unknown values rather than inferred
- Low-confidence visual classifications propagate uncertainty into downstream reasoning rather than forcing deterministic conclusions

This separation allows the system to reason over incomplete or uncertain information while preventing visual misclassifications from directly biasing domain-level threat assessments. This design enables modular updates to the knowledge base without requiring retraining of the visual classifier.

This design allows the reasoning layer to remain model-agnostic and extensible.

```
def ask_ollama(spec_text):
    prompt = f"""
You are a military aviation analyst.

From the provided information, produce STRICT YAML ONLY with this schema:

aircraft:
  name:
  role:
  country:
  introduction_year:

performance:
  max_speed:
    raw:
    kph:
  combat_range:
    raw:
    km:
  service_ceiling_m:

capabilities:
  stealth: true/false
  bvr_combat: true/false
  multirole: true/false
  strike: true/false
  electronic_warfare: true/false

sensors:
  radar:
    present: true/false
    type:
  eo_ir: true/false
  datalink: true/false

armament:
  air_to_air:
    - name
  air_to_ground:
    - name
  internal_bay: true/false

threat_assessment:
  level: Low/Medium/High
  rationale:
```

Figure 4: Ollama Prompt for YAML

Threat Reasoning Under Uncertainty

A core design assumption of this system is that **missing and imperfect information is the norm rather than the exception**. In this context, *uncertainty* refers to incomplete aircraft specifications, low-confidence visual classifications, or limited external documentation.

Specifications may be partially unavailable. Classification confidence may be low. Some aircraft types may be poorly documented or ambiguously defined.

Rather than failing or forcing deterministic outputs, the threat reasoning module is designed to operate explicitly under uncertainty. It:

- Tracks uncertainty originating from both visual classification confidence and missing specification fields
- Adjusts threat assessments conservatively when confidence is low or information is incomplete
- Produces human-readable explanatory summaries instead of opaque numerical scores

Threat levels are derived using interpretable, domain-informed heuristics such as aircraft speed, armament, and inferred mission profile, rather than learned black-box threat scores.

Each threat assessment consists of:

- A qualitative threat level
- A natural-language summary explaining the assessment
- Explicit uncertainty notes highlighting information gaps and confidence limitations

This presentation mirrors how human analysts communicate risk, emphasizing interpretability, caution, and transparency over false precision. This design prioritizes safe ambiguity over confident error in safety-critical decision contexts.

Action Recommendation Layer

The action recommendation layer operates on top of the threat assessment module. Its role is to **suggest potential next steps**, not to issue commands or make decisions. In this context, an *action recommendation* refers to a non-binding prompt intended to support human judgment.

This layer is strictly **ADVISORY ONLY** and does not override operator authority, rules of engagement, or command intent.

Recommendations are generated using doctrine-inspired, interpretable logic and are grouped into:

- Immediate actions (time-critical awareness and safety measures)
- Short-term actions (resource allocation and posture adjustments)
- Contingency preparations (planning for escalation or uncertainty)

Suggested actions are conditioned on assessed threat level, classification confidence, and available specification data. For example, a high-confidence, high-threat identification may result in recommendations such as continuous tracking, sensor fusion prioritization, or interceptor readiness.

When confidence is low or information is incomplete, recommendations are deliberately conservative and emphasize information gathering over escalation.

This design ensures the system functions as a decision-support aid, reinforcing situational awareness while preserving human control and accountability.

Code Used for Action Recommendation is in the Appendix section

Limitations

Despite its utility as a decision-support tool, this system has several inherent limitations that must be acknowledged. These limitations arise both from the nature of automated perception systems in operational settings and from constraints specific to the implementation presented in this work.

General System Limitations

- **Perceptual ambiguity:** Visual aircraft identification from a single image is inherently uncertain due to occlusion, viewing angle, lighting conditions, and visual similarity between aircraft families.
- **Data dependence:** Performance is fundamentally limited by the quality, diversity, and representativeness of available training data. Rare, classified, or newly introduced platforms may be misidentified or unrecognized.
- **Model uncertainty:** Confidence scores reflect statistical likelihood, not truth. High confidence does not guarantee correctness, and low confidence does not imply absence of threat.
- **Automation bias risk:** Even advisory systems may influence human judgment, particularly in time-sensitive environments, potentially biasing operators toward premature conclusions.
- **Lack of intent inference:** The system assesses capability, not intent. A highly capable aircraft does not necessarily pose an immediate threat, and intent must be inferred through additional contextual information.

Limitations Specific to This Implementation

- **Limited training data:** The classifier was trained on a small, narrowly scoped dataset, resulting in reduced generalization performance and an overall classification accuracy of approximately 41%.
- **Class imbalance and noise:** Certain aircraft classes contain limited or noisy samples, increasing the likelihood of misclassification and unstable confidence estimates.

- **Family-level confusion:** The model frequently confuses visually similar aircraft within the same design lineage (e.g., F-22 classified as F-15). While operationally informative, this reduces fine-grained identification reliability.
- **Incomplete specification coverage:** Aircraft specifications are derived from open-source material and may be incomplete, outdated, or inconsistent across platforms, leading to uncertainty in downstream threat reasoning.
- **Heuristic-based threat assessment:** Threat levels are computed using interpretable heuristics rather than learned intent models, which limits adaptability to novel operational contexts.
- **Single-frame inference:** The system operates on individual images and does not incorporate temporal tracking, multi-sensor fusion, or contextual battlefield information.

These limitations reinforce the necessity of human oversight and contextual reasoning. The system is best understood as an assistive analytical tool rather than an autonomous decision-maker.

Why This Matters

This project is not about achieving state-of-the-art accuracy.

It is about:

- Designing systems that reason under uncertainty
- Respecting the limits of machine perception
- Building pipelines that support humans, not replace them

The final system behaves less like a classifier and more like a junior analyst: confident when warranted, cautious when uncertain, and always explicit about its assumptions.

Next Steps

Potential future extensions include:

- **Temporal tracking across multiple frames:** Extending the system from single-image inference to temporal analysis would enable persistent tracking of aircraft across consecutive frames or sensor updates. This would allow the system to reason about motion patterns, trajectory changes, loitering behavior, and intent inference, reducing susceptibility to transient misclassifications and improving confidence stability over time.
- **Multi-aircraft fusion:** Supporting simultaneous detection and reasoning over multiple aircraft would allow the system to assess coordinated behavior, formation flying, and swarm-like activity. This is particularly relevant in contested airspace, where threat assessment depends not only on individual platforms but also on collective behavior and relative positioning.

- **Rules-of-engagement-aware recommendations:** Integrating configurable rules of engagement (ROE) using Retrieval Augmented Generation (RAG) systems would constrain threat assessments and action recommendations within predefined operational, legal, and political boundaries. This ensures that system outputs remain aligned with mission-specific doctrine and reduces the risk of inappropriate escalation suggestions.
- **Confidence-calibrated decision thresholds:** Incorporating calibrated confidence thresholds would allow the system to explicitly modulate its outputs based on uncertainty. Low-confidence classifications could trigger conservative recommendations or defer judgment, while high-confidence assessments could enable stronger advisory signals, reinforcing appropriate caution in ambiguous scenarios.

Ethical Implications

The integration of an automated identification and threat assessment system introduces significant ethical considerations centered on human judgment and accountability.

- **Automation Bias and Conformity:** Presenting a system-generated assessment creates **automation bias**, predisposing operators to conform to its suggestions. This can lead toward either unwarranted escalation or passive acceptance of potentially inaccurate judgments. This risk is amplified by the tendency of LLM-based components to align with user input, potentially reinforcing pre-existing biases rather than providing a neutral evaluation.
- **Desensitization and Decision Pressure:** Automating the threat-response loop can desensitize commanding officers to the gravity of their decisions, placing them under intense pressure to make rapid, system-validated calls. This environment risks eroding the crucial **tacit knowledge** and **command intuition** developed through experience.
- **The Fallibility Principle:** All automated systems are fallible. Therefore, the system must be designed strictly as a **decision-support tool**—a guiding hand that provides critical information without usurping the commander's ultimate responsibility. The human must remain firmly **in the decision loop** and **at the helm**.

Conclusion

This work presented an end-to-end aircraft identification and threat assessment pipeline designed to assist time-constrained decision-making in operational settings. By decoupling visual perception from domain knowledge, the system first classifies an observed aircraft and then reasons about its capabilities, threat level, and appropriate advisory actions using a structured specification database.

Rather than optimizing solely for classification accuracy, the system emphasizes interpretability, uncertainty awareness, and human-centric outputs. Classification confidence is explicitly propagated through downstream reasoning, incomplete specifications are handled gracefully, and threat assessments are expressed in explanatory summaries instead of opaque scores. This mirrors how real analysts communicate risk under uncertainty.

Experimental results demonstrate that while fine-grained identification accuracy is limited by data availability and class ambiguity, family-level recognition and capability inference remain operationally useful. Misclassifications between visually similar aircraft, such as within the same design lineage, still yield meaningful threat assessments, reinforcing the value of reasoning beyond the vision model alone.

Crucially, the system is designed as an advisory aid rather than an autonomous decision-maker. It does not infer intent, issue commands, or replace human judgment. Instead, it provides structured, confidence-aware guidance that can reduce cognitive load while preserving operator responsibility.

Overall, this project highlights that effective decision-support systems in safety-critical domains depend not only on model performance, but on transparency, uncertainty handling, and principled integration with human workflows. These considerations are essential for deploying AI systems responsibly in real-world operational environments.

Closing Thoughts

Beyond the technical results, this project highlighted the importance of systems-level thinking when deploying machine learning in safety-critical contexts.

A Appendix

```
    def recommend_actions(threat_level, confidence):
    """
    Returns recommended actions based on threat level and confidence.
    """

    actions = {
        "immediate": [],
        "short_term": [],
        "contingency": []
    }

    # --- Baseline monitoring ---
    actions["immediate"].append("Maintain continuous tracking")
    actions["immediate"].append("Cross-check with radar and IFF")

    if threat_level in ["Moderate", "High", "Severe"]:
        actions["short_term"].append("Increase sensor fusion priority")
        actions["short_term"].append("Assign interceptor-ready assets")

    if threat_level in ["High", "Severe"]:
        actions["short_term"].append("Elevate local alert posture")
        actions["contingency"].append("Prepare escalation protocols")

    if threat_level == "Severe":
        actions["contingency"].append("Notify command authority")
        actions["contingency"].append("Review rules of engagement")
```

```
27
28      # --- Confidence modifier ---
29      if confidence < 0.5:
30          actions["immediate"].append("Revalidate classification with
                additional sensors")
31
32      return actions
```

The complete code can be found in my github repository.

Link: https://github.com/JeevanLowrence/LYFT